

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 4

Развёртывание приложения на удалённом сервере

Выполнил:

Соболев Артём

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Провести предварительную настройку удалённого сервера (установка необходимых зависимостей)
2. Провести развёртывание приложения на удалённом сервере
3. Настроить nginx в режиме обратного прокси для обеспечения доступа к вашему приложению извне

Ход работы

В качестве удаленного сервера использовался VPS с Ubuntu Server 22.04. Подключение к серверу выполнялось по SSH. Перед развертыванием сервер был подготовлен: обновлены системные пакеты, установлены Docker, Docker Compose Plugin. Далее был установлен и запущен Nginx:

```
sudo apt install -y nginx
sudo systemctl enable nginx
sudo systemctl start nginx
```

Для ограничения внешнего доступа был настроен firewall ufw. Были открыты только необходимые порты:

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx HTTP'
sudo ufw allow 15672/tcp
sudo ufw enable
sudo ufw status
```

В результате наружу доступны:

1. 80 – HTTP-доступ через Nginx
2. 15672 – RabbitMQ Management UI

После подготовки сервера был клонирован репозиторий проекта. Для запуска приложения был подготовлен файл окружения .env. В файле

окружения были указаны параметры подключения к базам данных, JWT-секреты и учетные данные RabbitMQ. Приложение было запущено с помощью Docker Compose. После запуска была выполнена проверка контейнеров.

Для обеспечения внешнего доступа к приложению был настроен Nginx в режиме обратного прокси. Конфигурационный файл nginx-fitness-api.conf был скопирован в директорию /etc/nginx/sites-available/fitness-api и подключен через sites-enabled. Были выполнены команды:

```
cp nginx-fitness-api.conf /etc/nginx/sites-available/fitness-api
ln -s /etc/nginx/sites-available/fitness-api /etc/nginx/sites-enabled/fitness-api
rm -f /etc/nginx/sites-enabled/default
nginx -t
systemctl reload nginx
```

Nginx принимает внешние HTTP-запросы на 80 порту и перенаправляет их на соответствующие микросервисы:

```
/api/v1/auth/**, /api/v1/profile, /api/v1/users/** - user-service
/api/v1/videos/** - media-service
/api/v1/exercises/**, /api/v1/trainings/**, /api/v1/tags/**, /api/v1/types/** - training-service
/api/v1/articles/**, /api/v1/categories/**, /api/v1/comments/** - blog-service
/api/v1/progress/**, /api/v1/training/**, /api/v1/training-session-exercises/** - progress-service
```

После настройки Nginx приложение стало доступно извне через единый адрес.

Вывод

В ходе лабораторной работы был подготовлен удаленный сервер, установлены необходимые зависимости, развернуто микросервисное приложение через Docker Compose и настроен Nginx reverse proxy. Внешний доступ к приложению осуществляется через 80 порт по единому адресу, а Nginx перенаправляет запросы на соответствующие микросервисы. Таким образом, приложение успешно развернуто на удаленном сервере и доступно извне.